Low latency Java systems

Stefan Angelov

Architect Lead at nexo

Co-Founder and Trainer at #hacker4e_

Stefan Angelov

The Java guy from pLoVEdiv

You can find me here:

Agenda

- What is low latency systems?
- How JVM works (Phases & Metaspace)
- Java’s Memory Model — Heap, Stack, and Metaspace
- Heap Memory Best Practices
- Thread Synchronization
- LMAX disruptor
- Chronicle Map
- Chronicle Queue

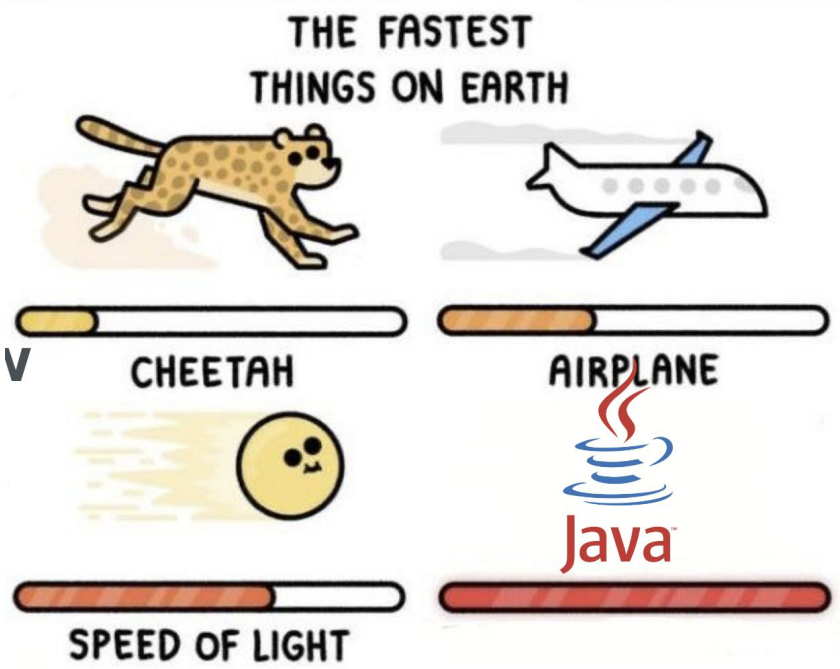
Is Java a good choice for low latency programing ?

What is low latency systems?

- Low latency systems are computer systems or networks engineered to minimize the time it takes to process and transmit data.
- They are optimized for speed, aiming to achieve the lowest possible delay or latency in performing tasks.

Introduction - Timescales

- 1 / 1 000 s = millisecond (ms)
- 1 / 1 000 000 s = microsecond (us)
- 1 / 1 000 000 000 s = nanosecond (ns)



Language comparison

- Java - aim for 10 us (microsecond)
- C/C++ aim for 10 us (microsecond)

Python is around 50 times slower than Java

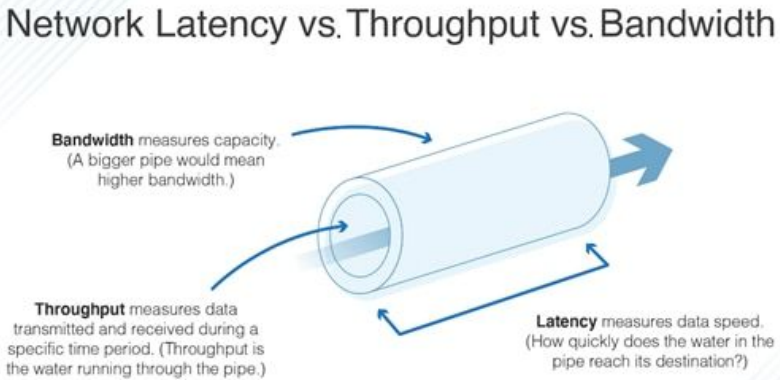
Low latency domains

- High frequency trading
- Online gaming
- Betting and bidding
- Vehicle autopilot systems

Throughput, Batching, and Response Time

Throughput Measurements

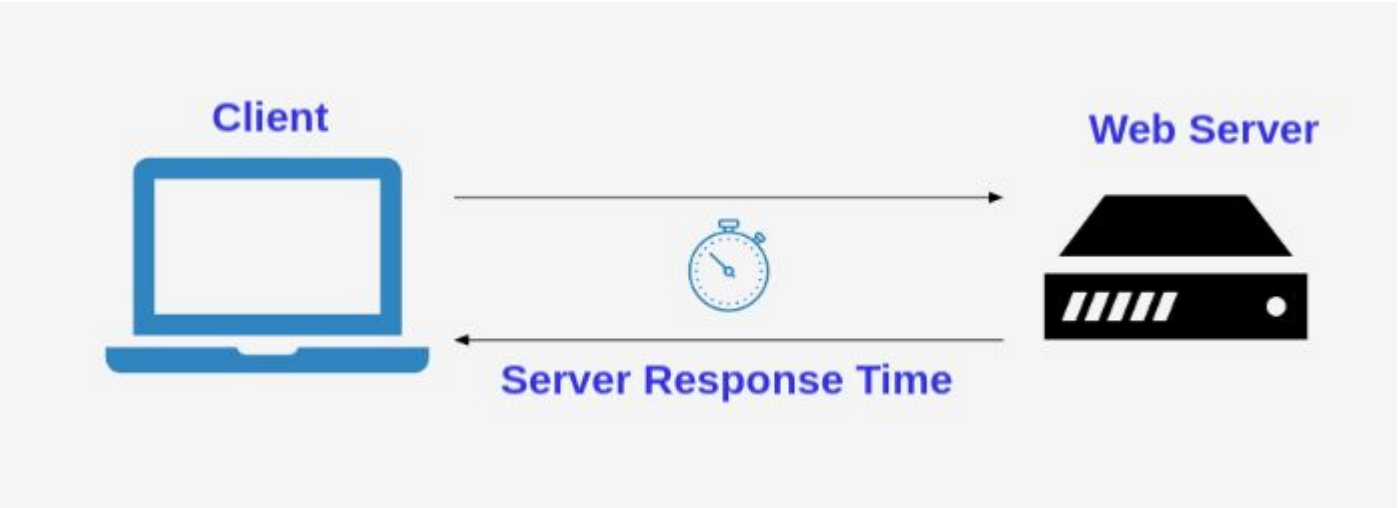
A throughput measurement is based on the amount of work that can be accomplished in a certain period of time



- Transactions per second (TPS)
- Requests per second (RPS)
- Operations per second (OPS)

Response-Time

Response time refers to the interval between the initiation of a request and the completion of the response.



Why Java not some other language ?

Wide ecosystems

Amazing Community

Speed of development

More smooth hiring

Speed of code

What are the challenges of using Java for low latency?

Challenges of using Java

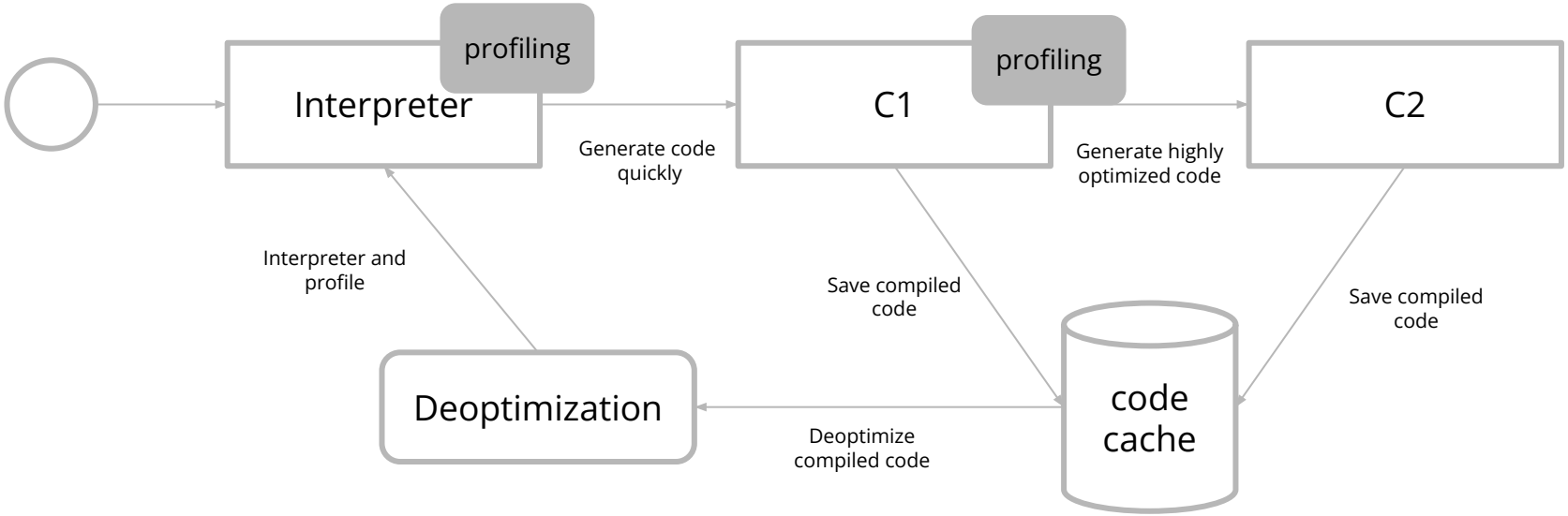
- Garbage collection
- Warmup
- Unpredictable compilation
- Don't have control of the memory layout
- Unsafe

Let’s try to understand how to mitigate some of those challenges...

How JVM works (Phases & Metaspace)

Just-in-Time Compilers: An Overview

Java applications are compiled,but instead of being compiled into a specific binary for a specific CPU, they are compiled into an intermediate low-level language. This language (known as Java bytecode) is then run by the java binary.



```
logger.fine("Welcome to JPrime with more that "+getNumberOfParticipants()+" participants", );
```

string concatenation

getNumberOfParticipants invocations

Are there any problems here?

```
if (logger.isLoggable(Level.FINE)) {
          logger.fine("Welcome to JPrime with more that {} participants", getNumberOfParticipants());
      }
```

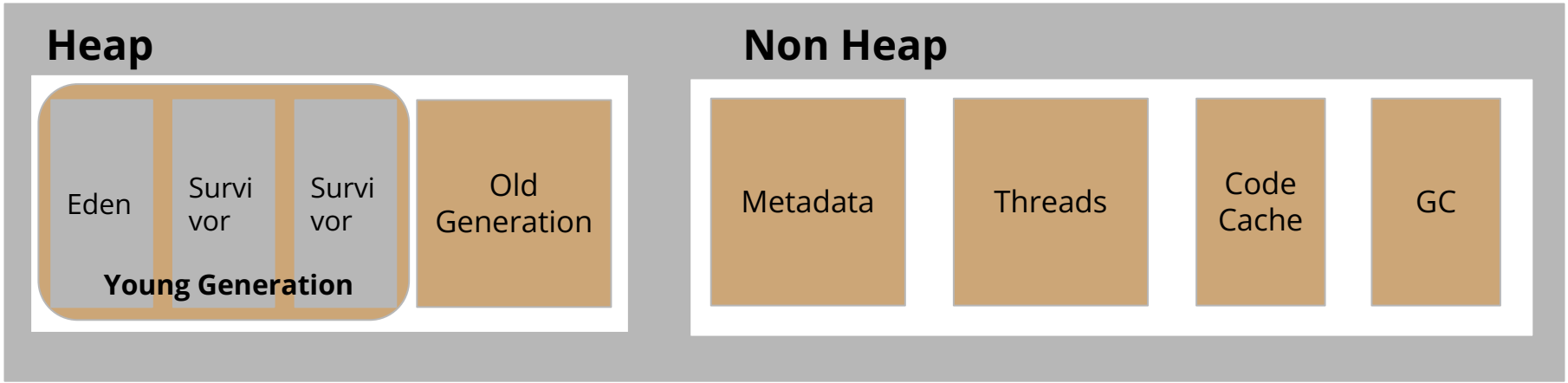
Registers and Main Memory

One of the most important optimizations a compiler can make involves when to use values from main memory and when to store values in a register. Consider this code:

```
ublic class RegisterTest {
    private int sum;
    public void calculateSum(int n) {
        for (int i = 0; i < n; i++) {
            sum += i;
        }
    }
```

Latency from CPU to	CPU cycles	Time
Main memory	Multiple	~60-80 ns
L3 cache	~40-45 cycles	~15 ns
L2 cache	~10 cycles	~3 ns
L1 cache	~3-4 cycles	~1 ns
Register	1 cycle	Very very quick

Java’s Memory Model — Heap, Stack, and Metaspace



Heap Memory Best Practices

Using Less Memory

The first approach to using memory more efficiently in Java is to use less heap memory. That statement should be unsurprising: using less memory means the heap will fill up less often, requiring fewer GC cycles. The effect can multiply: fewer collections of the young generation means the tenuringa age of an object is increased less often meaning that the object is less likely to be promoted into the old generation.

Reducing Object Size

Type	Size
byte	1
char	2
short	2
int	4
float	4
long	8
double	8
reference	8

The size of an object can be decreased by (obviously) reducing the number of instance variables it holds and (less obviously) by reducing the size of those variables.

```
public class A {
    private int i;
}
public class B {
    private int i;
    private Locale l = Locale.US;
}
public class C {
    private int i;
    private ConcurrentHashMap chm = new ConcurrentHashMap();
}
```

Using Lazy Initialization

```
public class CalDateInitialization {
     private Calendar calendar;
     private DateFormat df;

    private void report(Writer w) {
        if (calendar == null) {
                calendar = Calendar.getInstance();
                df = DateFormat.getDateInstance();
        }
        w.write("On " + df.format(calendar.getTime()) + ": " + this);
    }
}
```

Using Immutable

In Java, many object types are immutable. This includes objects that have a corresponding primitive type—Integer, Double, Boolean, and so on—as well as other numeric-based types, like BigDecimal. The most common Java object, of course, is the immutable String. From a program design perspective, it is often a good idea for custom classes to represent immutable objects as well.

```
Boolean.TRUE;
Bollean.FALSE;
```

These singular representations of immutable objects are known as the canonical version of the object.

Canonical Objects

```
public class ImmutableObject {
    private static WeakHashMap<String, WeakReference<ImmutableObject>> map = new WeakHashMap();
    public ImmutableObject canonicalVersion(String io) {
        synchronized(map) {
            ImmutableObject canonicalVersion = map.get(io).get();
            if (canonicalVersion == null) {
                canonicalVersion = new ImmutableObject();
                map.put(io, new WeakReference<>(canonicalVersion));
            }
            return canonicalVersion;
        }
    }
}
```

Object Reuse

Object reuse is commonly achieved in two ways: object pools and thread-local variables. Object pooling, in particular, is widely disliked in GC circles for that reason, though for that matter, object pools are also widely disliked in development circles for many other reasons.

Object Pools - Code

Thread Local - Code

Singleton design pattern - Code

Flyweight

Flyweight is a structural design pattern that allows programs to support vast quantities of objects by keeping their memory consumption low. The pattern achieves it by sharing parts of object state between multiple objects. In other words, the Flyweight saves RAM by caching the same data used by different objects.

Using primitive type collections

There is a couple of libraries providing the collections where primities could be stored instead of wrappers

- https://github.com/real-logic/agrona
- https://github.com/eclipse/eclipse-collections

Using primitive type collections

There is a couple of libraries providing the collections where primities could be stored instead of wrappers

- https://github.com/real-logic/agrona
- https://github.com/eclipse/eclipse-collections

Thread Synchronization

Costs of Synchronization

Synchronized areas of code affect performance in two ways. First, the amount of time an application spends in a synchronized block affects the scalability of an application. Second, obtaining the synchronization lock requires CPU cycles and hence affects performance.

- Context Switching
- Contention
- Lock Acquisition and Release
- Memory Consistency Effects

Avoiding Synchronization

If synchronization can be avoided altogether, locking penalties will not affect the application’s performance. Two general approaches can be used to achieve that.

- Thread-local variable
- CAS-based alternatives

Thread-local variable

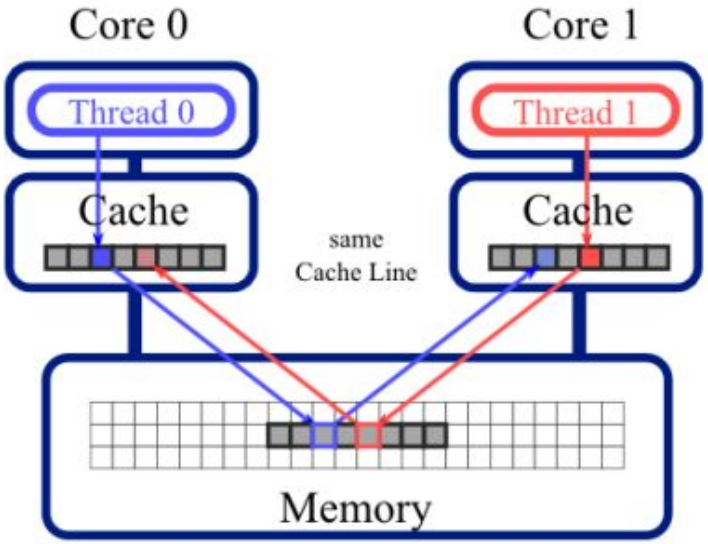
```
public class Thermometer {
    private static ThreadLocal<NumberFormat> nfLocal = new ThreadLocal<>() {
        public NumberFormat initialValue() {
            NumberFormat nf = NumberFormat.getInstance();
             nf.setMinumumIntegerDigits(2);
            return nf;
        }
    }
    public String toString() {
         NumberFormat nf = nfLocal.get();
          nf.format(...);
          return nf.toString();
    }
}
```

CAS-based alternatives

```
AtomicLong al = new AtomicLong(0);
 public long doOperation() {
 return al.getAndIncrement(); }
```

False sharing

Synchronized areas of code affect performance in two ways. First, the amount of time an application spen in a synchronized block affects the scalability of an application. Second, obtaining the synchronization lock requires CPU cycles and hence affec performance.



False sharing

Synchronized areas of code affect performance in two ways. First, the amount of time an application spends in a synchronized block affects the scalability of an application. Second, obtaining the synchronization lock requires CPU cycles and hence affects performance.

```
public class DataHolder {
    public volatile long l1;
    public volatile long l2;
    public volatile long l3;
    public volatile long l4;
}
```

Thread Affinity

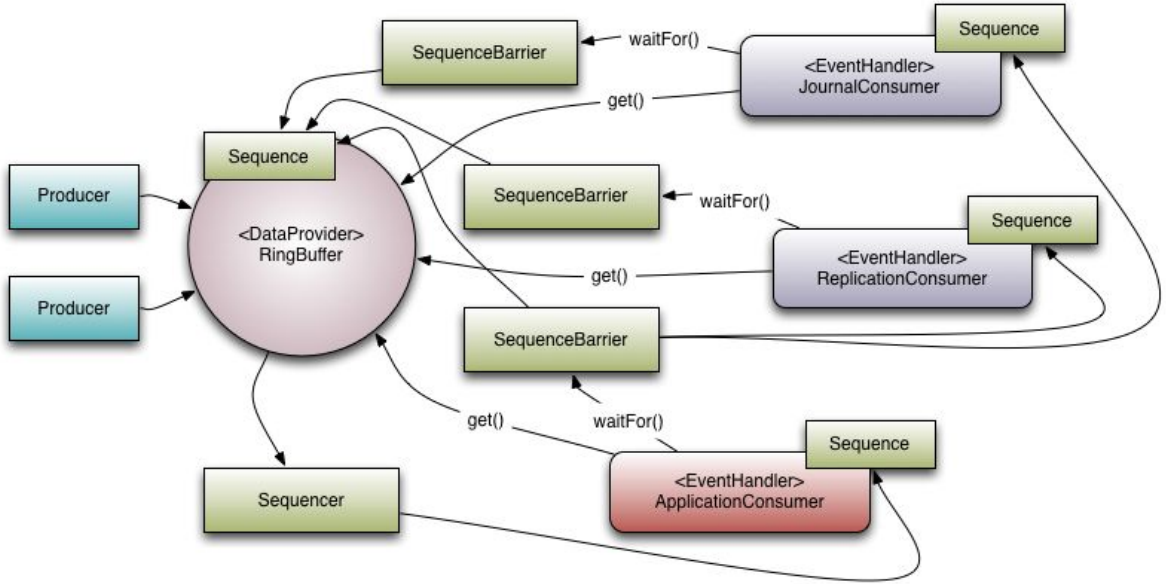
Thread affinity is the concept of mapping a thread to a specific processor or core in a multi-core system. When a thread is bound to a specific processor or core, it can only be executed on that processor or core. This can help to reduce cache misses, increase data locality, and reduce context switching overhead.

```
try (AffinityLock al = AffinityLock.acquireCore()) {
    // do some work while locked to a CPU.
}
```

LMAX distruptor

LMAX Disruptor

The LMAX Disruptor is a high performance inter-thread messaging library. It grew out of LMAX's research into concurrency, performance and non-blocking algorithms and today forms a core part of their Exchange's infrastructure.



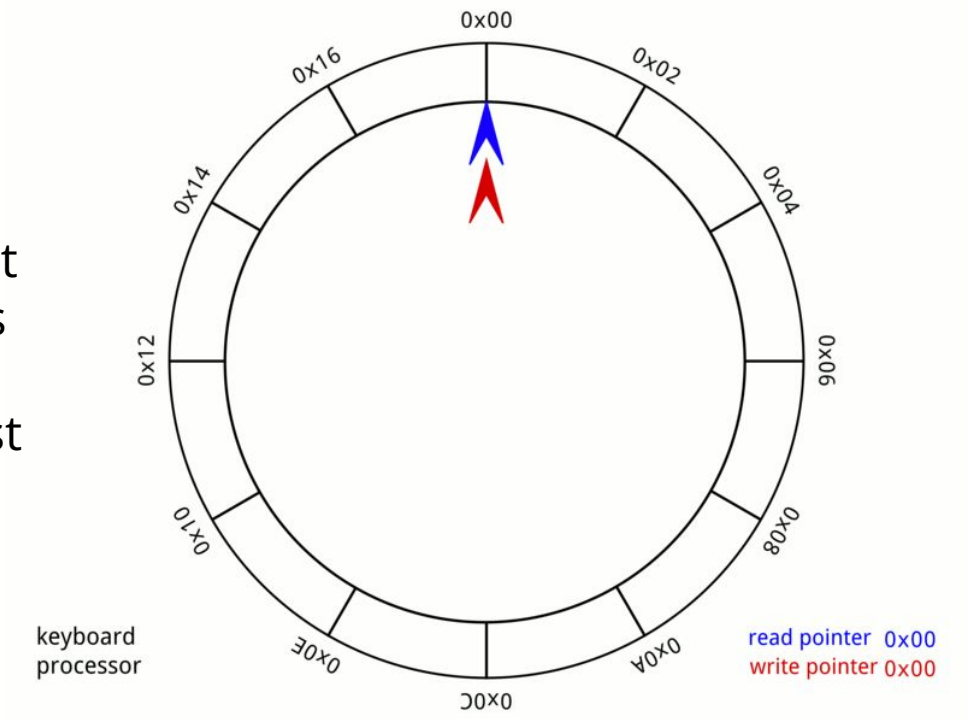
LMAX Disruptor

To understand the benefits of the Disruptor we can compare it to something well understood and quite similar in purpose. In the case of the Disruptor this would be Java's BlockingQueue. Like a queue the purpose of the Disruptor is to move data (e.g. messages or events) between threads within the same process. However, there are some key features that the Disruptor provides that distinguish it from a queue. They are:

- Multicast events to consumers, with consumer dependency graph.
- Pre-allocate memory for events.
- Optionally lock-free.

Ring Buffer

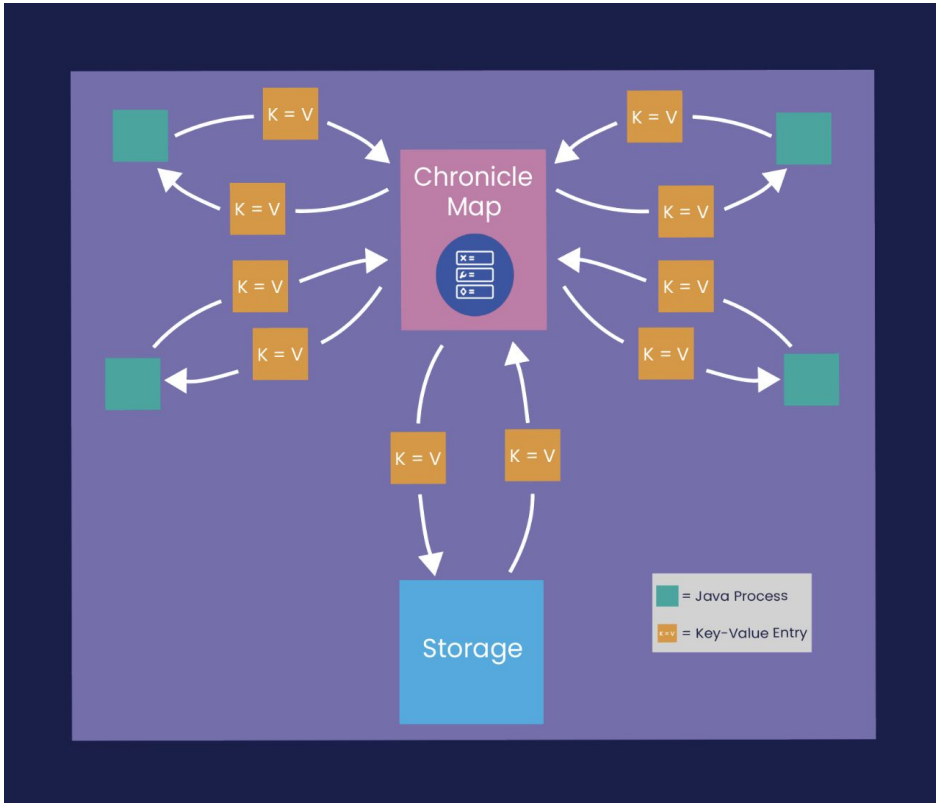
The ring buffer is a circular software queue. This queue has a first-in-first-out (FIFO) data characteristic. These buffers are quite common and are found in many embedded systems. Usually, most developers write these constructs from scratch on an as-needed basis.



Chronicle Map

Chronicle Map is a super-fast, in-memory, non-blocking, key-value store, designed for low-latency, and/or multi-process applications such as trading and financial market applications. See Features doc for more information.

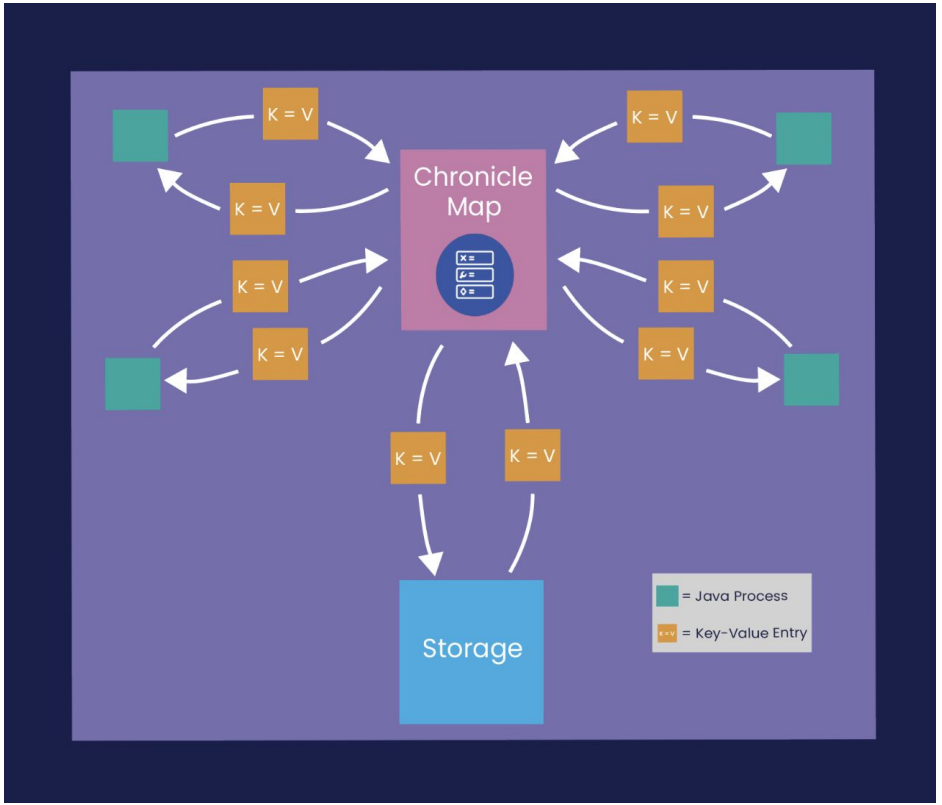
The size of a Chronicle Map is not limited by memory (RAM), but rather by the available disk capacity.



Chronicle Map

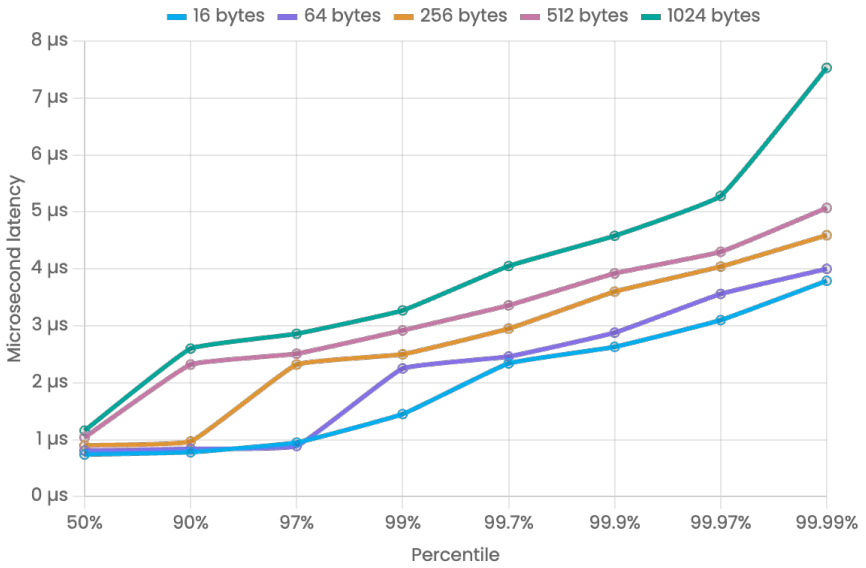
Use cases:

- Real-time trading systems
- highly concurrent systems



Chronicle Queue

Chronicle Queue can be considered to be similar to a low-latency, broker-less durable/persisted JVM topic. It is a distributed, unbounded, persisted queue that:



- Supports asynchronous RMI and pub/sub interfaces with microsecond latencies
- Passes messages between JVMs in under 1 µs*
- Provides stable, soft, real-time latencies into the millions of messages/s for a single thread to one queue

Is Java a good choice for low latency programing ?

Resources:

- https://www.oreilly.com/library/view/java-performance-2nd/9781492056102/
- https://chronicle.software/chronicle-tune-your-cpu/
- https://chronicle.software/chronicle-tune-your-cpu/
- https://chronicle.software/java-is-very-fast/
- https://blog.stackademic.com/optimizing-java-for-low-latency-applications-803e181d0add
- https://www.youtube.com/watch?v=36t5OLLzA6c
- https://www.youtube.com/watch?v=BD9cRbxWQx8